JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ThreadGroup

java.lang.Object
    java.lang.ThreadGroup

All Implemented Interfaces:

`Thread.UncaughtExceptionHandler`

```
public class ThreadGroup
extends Object
implements Thread.UncaughtExceptionHandler
```

A thread group represents a set of threads. In addition, a thread group can also include other thread groups. The thread groups form a tree in which every thread group except the initial thread group has a parent.

A thread group has a name and maximum priority. The name is specified when creating the group and cannot be changed. The group's maximum priority is the maximum priority for threads created in the group. It is initially inherited from the parent thread group but may be changed using the `setMaxPriority` method.

A thread group is weakly *reachable* from its parent group so that it is eligible for garbage collection when there are no live threads in the group and the thread group is otherwise *unreachable*.

Unless otherwise specified, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

Thread groups and virtual threads

The Java runtime creates a special thread group for virtual threads. This group is returned by the `Thread.getThreadGroup` method when invoked on a virtual thread. The thread group differs to other thread groups in that its maximum priority is fixed and cannot be changed with the `setMaxPriority` method. Virtual threads are not included in the estimated thread count returned by the `activeCount` method, are not enumerated by the `enumerate` method, and are not interrupted by the `interrupt` method.

API Note:

> Thread groups provided a way in early Java releases to group threads and provide a form of *job control* for threads. Thread groups supported the isolation of applets and defined methods intended for diagnostic purposes. It should be rare for new applications to create ThreadGroups and interact with this API.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
`ThreadGroup(String name)`	Constructs a new thread group.
`ThreadGroup(ThreadGroup parent, String name)`	Creates a new thread group.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`int`	`activeCount()`	Returns an estimate of the number of live platform threads in this thread group and its subgroups.
`int`	`activeGroupCount()`	Returns an estimate of the number of groups in this thread group and its subgroups.
`final void`	`checkAccess()`	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally determined if the currently running thread had permission to modify this thread group.
`final void`	`destroy()`	**Deprecated, for removal: This API element is subject to removal in a future version.**

		This method was originally specified to destroy an empty thread group.
int	**enumerate**(Thread[] list)	Copies into the specified array every live platform thread in this thread group and its subgroups.
int	**enumerate**(Thread[] list, boolean recurse)	Copies into the specified array every live platform thread in this thread group.
int	**enumerate**(ThreadGroup[] list)	Copies into the specified array references to every subgroup in this thread group and its subgroups.
int	**enumerate**(ThreadGroup[] list, boolean recurse)	Copies into the specified array references to every subgroup in this thread group.
final int	**getMaxPriority**()	Returns the maximum priority of this thread group.
final String	**getName**()	Returns the name of this thread group.
final ThreadGroup	**getParent**()	Returns the parent of this thread group.
final void	**interrupt**()	Interrupts all live platform threads in this thread group and its subgroups.

final boolean	**isDaemon**()	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally indicated if the thread group is a *daemon thread group* that is automatically destroyed when its last thread terminates.
boolean	**isDestroyed**()	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally indicated if the thread group is destroyed.
void	**list**()	Prints information about this thread group to the standard output.
final boolean	**parentOf**(ThreadGroup g)	Tests if this thread group is either the thread group argument or one of its ancestor thread groups.
final void	**setDaemon** (boolean daemon)	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally configured whether the thread group is a *daemon thread group* that is automatically destroyed when its last thread terminates.

final void	**setMaxPriority**(int pri)	Sets the maximum priority of the group.
String	**toString**()	Returns a string representation of this Thread group.
void	**uncaughtException**(Thread t, Throwable e)	Called by the Java Virtual Machine when a thread in this thread group stops because of an uncaught exception, and the thread does not have a specific Thread.UncaughtExceptionHandler installed.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

ThreadGroup

```
public ThreadGroup(String name)
```

Constructs a new thread group. The parent of this new group is the thread group of the currently running thread.

Parameters:

name - the name of the new thread group, can be null

ThreadGroup

```
public ThreadGroup(ThreadGroup parent,
                   String name)
```

Creates a new thread group. The parent of this new group is the specified thread group.

Parameters:

parent - the parent thread group.

name - the name of the new thread group, can be null

Method Details

getName

```
public final String getName()
```

Returns the name of this thread group.

Returns:

the name of this thread group, may be `null`

getParent

```
public final ThreadGroup getParent()
```

Returns the parent of this thread group.

Returns:

the parent of this thread group. The top-level thread group is the only thread group whose parent is `null`.

getMaxPriority

```
public final int getMaxPriority()
```

Returns the maximum priority of this thread group. This is the maximum priority for new threads created in the thread group.

Returns:

the maximum priority for new threads created in the thread group

See Also:

`setMaxPriority(int)`

isDaemon

```
@Deprecated(since="16",
            forRemoval=true)
public final boolean isDaemon()
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally indicated if the thread group is a daemon thread group that is automatically destroyed when its last thread terminates. The concept of daemon thread group no longer exists. A thread group is eligible to be GC'ed when there are no live threads in the group and it is otherwise unreachable.

Returns the daemon status of this thread group. The daemon status is not used for anything.

Returns:

the daemon status of this thread group

isDestroyed

```
@Deprecated(since="16",
            forRemoval=true)
public boolean isDestroyed()
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally indicated if the thread group is destroyed. The ability to destroy a thread group and the concept of a destroyed thread group no longer exists. A thread group is eligible to be GC'ed when there are no live threads in the group and it is otherwise unreachable.

Returns false.

Returns:

false

Since:

1.1

setDaemon

```
@Deprecated(since="16",
            forRemoval=true)
public final void setDaemon(boolean daemon)
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally configured whether the thread group is a daemon thread group that is automatically destroyed when its last thread terminates. The concept of daemon thread group no longer exists. A thread group is eligible to be GC'ed when there are no live threads in the group and it is otherwise unreachable.

Sets the daemon status of this thread group. The daemon status is not used for anything.

Parameters:

`daemon` - the daemon status

setMaxPriority

```
public final void setMaxPriority(int pri)
```

Sets the maximum priority of the group. The maximum priority of the *ThreadGroup for virtual threads* is not changed by this method (the new priority is ignored). Threads in the thread group (or subgroups) that already have a higher priority are not affected by this method.

If the `pri` argument is less than `Thread.MIN_PRIORITY` or greater than `Thread.MAX_PRIORITY`, the maximum priority of the group remains unchanged.

Otherwise, the priority of this ThreadGroup object is set to the smaller of the specified `pri` and the maximum permitted priority of the parent of this thread group. (If this thread group is the system thread group, which has no parent, then its maximum priority is simply set to `pri`.) Then this method is called recursively, with `pri` as its argument, for every thread group that belongs to this thread group.

Parameters:

`pri` - the new priority of the thread group.

See Also:

`getMaxPriority()`

parentOf

```
public final boolean parentOf(ThreadGroup g)
```

Tests if this thread group is either the thread group argument or one of its ancestor thread groups.

Parameters:

`g` - a thread group, can be `null`

Returns:

`true` if this thread group is the thread group argument or one of its ancestor thread groups; `false` otherwise.

checkAccess

```
@Deprecated(since="17",
            forRemoval=true)
public final void checkAccess()
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally determined if the currently running thread had permission to modify this thread group. This method was only useful in conjunction with the Security Manager, which is no longer supported. There is no replacement for the Security Manager or this method.

Does nothing.

activeCount

```
public int activeCount()
```

Returns an estimate of the number of live platform threads in this thread group and its subgroups. Virtual threads are not included in the estimate. This method recursively iterates over all subgroups in this thread group.

The value returned is only an estimate because the number of threads may change dynamically while this method traverses internal data structures, and might be affected by the presence of certain system threads. This method is intended primarily for debugging and monitoring purposes.

Returns:

an estimate of the number of live threads in this thread group and in any other thread group that has this thread group as an ancestor

enumerate

```
public int enumerate(Thread[] list)
```

Copies into the specified array every live platform thread in this thread group and its subgroups. Virtual threads are not enumerated by this method.

An invocation of this method behaves in exactly the same way as the invocation

```
enumerate(list, true)
```

Parameters:

`list` - an array into which to put the list of threads

Returns:

the number of threads put into the array

enumerate

```
public int enumerate(Thread[] list,
                     boolean recurse)
```

Copies into the specified array every live platform thread in this thread group. Virtual threads are not enumerated by this method. If `recurse` is `true`, this method recursively enumerates all subgroups of this thread group and references to every live platform thread in these subgroups are also included. If the array is too short to hold all the threads, the extra threads are silently ignored.

An application might use the activeCount method to get an estimate of how big the array should be, however *if the array is too short to hold all the threads, the extra threads are silently ignored.* If it is critical to obtain every live thread in this thread group, the caller should verify that the returned int value is strictly less than the length of `list`.

Due to the inherent race condition in this method, it is recommended that the method only be used for debugging and monitoring purposes.

Parameters:

`list` - an array into which to put the list of threads

`recurse` - if `true`, recursively enumerate all subgroups of this thread group

Returns:

the number of threads put into the array

activeGroupCount

```
public int activeGroupCount()
```

Returns an estimate of the number of groups in this thread group and its subgroups. Recursively iterates over all subgroups in this thread group.

The value returned is only an estimate because the number of thread groups may change dynamically while this method traverses internal data structures. This method is intended primarily for debugging and monitoring purposes.

Returns:

the number of thread groups with this thread group as an ancestor

enumerate

```
public int enumerate(ThreadGroup[] list)
```

Copies into the specified array references to every subgroup in this thread group and its subgroups.

An invocation of this method behaves in exactly the same way as the invocation

```
enumerate(list, true)
```

Parameters:

`list` - an array into which to put the list of thread groups

Returns:

the number of thread groups put into the array

enumerate

```
public int enumerate(ThreadGroup[] list,
                     boolean recurse)
```

Copies into the specified array references to every subgroup in this thread group. If `recurse` is `true`, this method recursively enumerates all subgroups of this thread group and references to every thread group in these subgroups are also included.

An application might use the activeGroupCount method to get an estimate of how big the array should be, however *if the array is too short to hold all the thread groups, the extra thread groups are silently ignored.* If it is critical to obtain every subgroup in this thread group, the caller should verify that the returned int value is strictly less than the length of `list`.

Due to the inherent race condition in this method, it is recommended that the method only be used for debugging and monitoring purposes.

Parameters:

`list` - an array into which to put the list of thread groups

`recurse` - if `true`, recursively enumerate all subgroups

Returns:

the number of thread groups put into the array

interrupt

```
public final void interrupt()
```

Interrupts all live platform threads in this thread group and its subgroups.

Since:

1.2

See Also:

`Thread.interrupt()`

destroy

```
@Deprecated(since="16",
            forRemoval=true)
public final void destroy()
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method was originally specified to destroy an empty thread group. The ability to explicitly destroy a thread group no longer exists. A thread group is eligible to be GC'ed when there are no live threads in the group and it is otherwise unreachable.

Does nothing.

list

```
public void list()
```

Prints information about this thread group to the standard output. This method is useful only for debugging.

uncaughtException

```
public void uncaughtException(Thread t,
                              Throwable e)
```

Called by the Java Virtual Machine when a thread in this thread group stops because of an uncaught exception, and the thread does not have a specific `Thread.UncaughtExceptionHandler` installed.

The `uncaughtException` method of `ThreadGroup` does the following:

- If this thread group has a parent thread group, the `uncaughtException` method of that parent is called with the same two arguments.
- Otherwise, this method checks to see if there is a default uncaught exception handler installed, and if so, its `uncaughtException` method is called with the same two arguments.
- Otherwise, a message containing the thread's name, as returned from the thread's `getName` method, and a stack backtrace, using the `Throwable`'s `printStackTrace` method, is printed to the standard error stream.

Applications can override this method in subclasses of `ThreadGroup` to provide alternative handling of uncaught exceptions.

Specified by:

`uncaughtException` in interface `Thread.UncaughtExceptionHandler`

Parameters:

`t` - the thread that is about to exit.

`e` - the uncaught exception.

toString

```
public String toString()
```

Returns a string representation of this Thread group.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this thread group.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.